

fallaría para los valores que no están presentes. Si un grafo incluye uno o más nodos con un número infinito de vecinos, pero sin caminos de longitud infinita, la búsqueda en profundidad puede tener éxito aun cuando falle el recorrido en anchura. Sin embargo, esta situación parece producirse con menos frecuencia que la inversa.

9.6 VUELTA ATRÁS

Tal como vimos anteriormente, hay distintos problemas que se pueden concebir en términos de grafos abstractos. Por ejemplo, veíamos en la Sección 9.1 que se pueden utilizar los nodos de un grafo para representar las situaciones de un juego de ajedrez, y las aristas para representar jugadas válidas. Con frecuencia, el problema original se traduce a buscar un nodo, un camino o un patrón específicos en el grafo asociado. Si el grafo contiene un número elevado de nodos, y especialmente si es infinito, puede resultar inútil o inabordable construirlo explícitamente en la memoria de una computadora antes de aplicar una de las técnicas de búsqueda que hemos empleado hasta el momento.

En tales situaciones utilizaremos un *grafo implícito*. Éste es un grafo para el cual se dispone de una descripción de sus nodos y aristas, de tal manera que se pueden construir partes relevantes del grafo a medida que progresa el recorrido. De esta manera, se ahorra tiempo de computación siempre que el recorrido tenga éxito antes de haber construido todo el grafo. La economía de espacio en memoria también puede ser dramática, sobre todo cuando se pueden descartar los nodos que hayan sido examinados ya, creando así espacio para poder explorar los nodos subsiguientes. Si el grafo en cuestión es infinito, las técnicas como ésta constituyen nuestra única esperanza de llegar a explorarlo. Esta sección, y las que siguen, detallan algunas formas estándar de organizar recorridos en un grafo implícito.

En su forma básica, la vuelta atrás se asemeja a un recorrido en profundidad dentro de un grafo dirigido. El grafo en cuestión suele ser un árbol, o por lo menos no contiene ciclos. Sea cual sea su estructura, existe sólo implícitamente. El objetivo del recorrido es encontrar soluciones para algún problema. Esto se consigue construyendo soluciones parciales a medida que progresa el recorrido; estas soluciones parciales limitan las regiones en las que se puede encontrar una solución completa. El recorrido tiene éxito si, procediendo de esta forma, se puede definir por completo una solución. En este caso el algoritmo puede bien detenerse (si lo único que se necesita es una solución del problema) o bien seguir buscando soluciones alternativas (si deseamos examinarlas todas). Por otra parte, el recorrido no tiene éxito si en alguna etapa la solución parcial construida hasta el momento no se puede completar. En tal caso, el recorrido vuelve atrás exactamente igual que en un recorrido en profundidad, eliminando sobre la marcha los elementos que se hubieran añadido en cada fase. Cuando vuelve a un nodo que tiene uno o más vecinos sin explorar, prosigue el recorrido de una solución.

9.6.1 El problema de la mochila (3)

Como primer ejemplo para ilustrar el principio general, volvamos al problema de la mochila que se describía en la Sección 8.4. Recordemos que se nos dan un cierto número de objetos y una mochila. En esta ocasión, sin embargo, en lugar de suponer que están disponibles n objetos, supondremos que lo que tenemos son n tipos de objetos, y que está disponible un número adecuado de objetos de cada tipo. Esto no altera el problema de forma sustancial. Para $i = 1, 2, \dots, n$ un objeto de tipo i tiene un peso positivo w_i y un valor positivo v_i . La mochila puede llevar un peso que no exceda de W . Nuestro objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos incluidos, respetando al mismo tiempo la restricción de capacidad. Podemos tomar un objeto o prescindir de él, pero no podemos tomar una cierta parte de un objeto.

Supongamos para concretar que deseamos resolver un caso del problema con cuatro tipos de objetos, cuyos pesos son respectivamente 2, 3, 4 y 5 unidades, y cuyos valores son 3, 5, 6 y 10 respectivamente. La mochila puede llevar un máximo de 8 unidades de peso. Esto se puede hacer utilizando la técnica de vuelta atrás, explorando el árbol implícito de la figura 9.12.

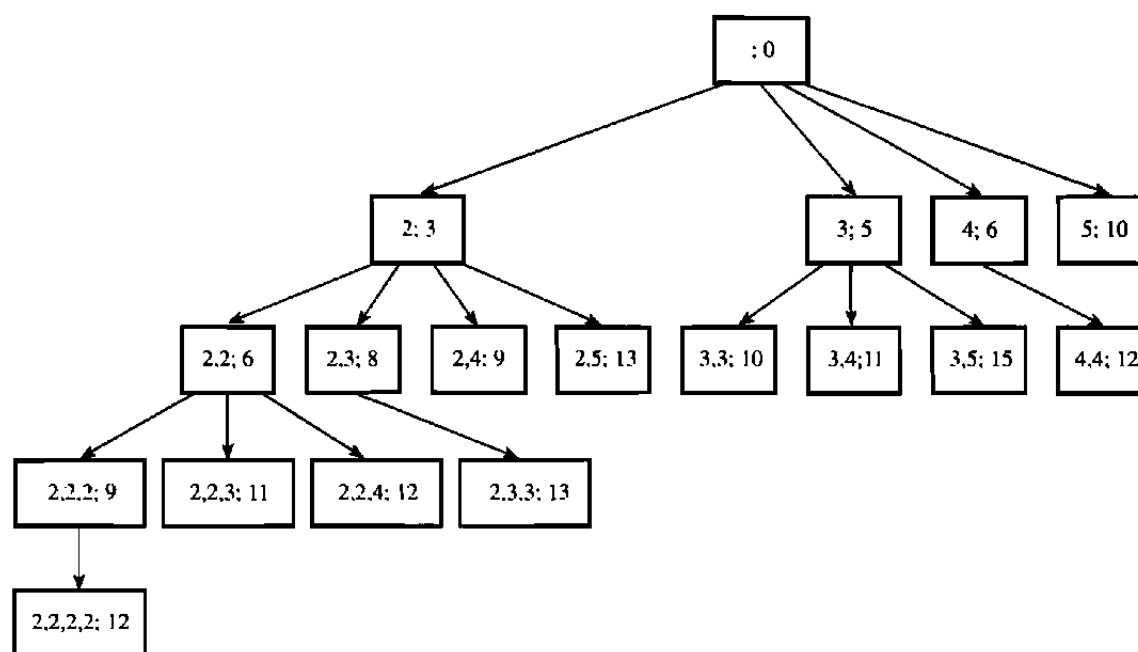


Figura 9.12. El árbol implícito para un problema de mochila

Aquí, un nodo tal como el (2, 3; 8) corresponde a una solución parcial de nuestro problema. Las cifras que están a la izquierda del punto y coma son los pesos de los objetos que hemos decidido incluir, y la cifra que está a la derecha es el valor actual de la carga. Bajar de un nodo a uno de sus hijos se corresponde con decidir qué clase de objeto se va a poner en la mochila a continuación. Sin pérdida de generalidad, podemos acordar que cargaremos los objetos en la mochila por orden creciente de peso. Esto no es esencial, y a decir verdad cualquier otro orden —por orden decre-

cienta de pesos, por ejemplo, o por valores— funcionaría igual de bien, pero así se reduce el tamaño del árbol que hay que explorar. Por ejemplo, una vez visitado el nodo (2, 2, 3; 11) no tiene sentido visitar posteriormente el (2, 3, 2; 11).

Inicialmente, la solución parcial está vacía. El algoritmo de vuelta atrás explora el árbol como un recorrido en profundidad, construyendo nodos y soluciones parciales a medida que avanza. En el ejemplo, el primer nodo que se visita es el (2; 3), el siguiente es el (2, 2; 6), el tercero es (2, 2, 2; 9) y el cuarto es (2, 2, 2, 2; 12). A medida que se visita cada nodo nuevo, se extiende la solución parcial. Después de visitar estos cuatro nodos, se bloquea el recorrido en profundidad: el nodo (2, 2, 2, 2; 12) no posee sucesores no visitados (de hecho, carece de sucesores), puesto que añadir más elementos a esta solución parcial violaría la restricción de capacidad. Dado que esta solución parcial puede resultar ser la solución óptima de nuestro caso, la memorizamos.

El algoritmo de recorrido en profundidad vuelve atrás ahora en busca de otras soluciones. En cada paso retrocedido por el árbol, se elimina el elemento correspondiente de la solución parcial. En el ejemplo, el recorrido vuelve primero a (2, 2, 2; 9), que también carece de sucesores no visitados; sin embargo, al retroceder un paso más por el árbol hasta el nodo (2, 2; 6), hay dos sucesores que quedan por visitar. Después de explorar los nodos (2, 2, 3; 11) y (2, 2, 4; 12), ninguno de los cuales mejora la solución memorizada anteriormente, la búsqueda retrocede otro paso más, y así sucesivamente. Explorando el árbol de esta manera, encontramos que (2, 3, 3; 13) es una solución mejor que la que ya tenemos, y después (3, 5; 15) es todavía mejor. Puesto que no se hace ninguna otra mejora antes de finalizar la búsqueda, ésta es la solución óptima del caso.

Programar el algoritmo es sencillo e ilustra la íntima relación existente entre recursión y recorrido en profundidad. Supongamos que los valores de n y de W , y los valores de la matrices $w[1..n]$ y de $v[1..n]$ correspondientes al caso que hay que resolver están disponibles como variables globales. La ordenación de los tipos de elementos es irrelevante. Se define una función *mochilava* de la forma siguiente:

```
función mochilava( $i, r$ )
    {Calcula el valor de la mejor carga que se puede
     construir empleando elementos de los tipos  $i$  a  $n$ 
     y cuyo peso total no sobrepase  $r$ }
     $b \leftarrow 0$ 
    {Se prueban por turno las clase de objetos admisibles }
    para  $k \leftarrow i$  hasta  $n$  hacer
        si  $w[k] \leq r$  entonces
             $b \leftarrow \max(b, v[k] + \textit{mochilava}(k, r - w[k]))$ 
    devolver  $b$ 
```

Ahora, para hallar el valor de la mejor carga, llamamos a *mochilava*(1, W). Cada llamada recursiva a *mochilava* se corresponde con extender el recorrido en profundidad hasta el nivel inmediatamente inferior del árbol, mientras que el bucle **para** se encarga de examinar todas las posibilidades en un nivel dado. En esta ver-

sión del programa, la composición de la carga que se está examinando está dada implícitamente por los valores de k en la pila recursiva. No es difícil adaptar el programa para que nos dé explícitamente la composición de la mejor carga junto con su valor; véase el Problema 9.42.

9.6.2 El problema de las ocho reinas

Para nuestro segundo ejemplo de vuelta atrás, consideremos el problema clásico consistente en situar las ocho reinas en un tablero de ajedrez de tal modo que ninguna de ellas amenace a ninguna de las demás. Recordemos que una reina amenaza a los cuadrados de la misma fila, de la misma columna o de las mismas diagonales.

La forma más evidente de resolver este problema consiste en probar sistemáticamente todas las formas de colocar ocho reinas en un tablero, determinando en cada ocasión si se ha encontrado una solución. Esta aproximación no tiene aplicación práctica, ni siquiera con una computadora, porque el número de situaciones que habría que examinar es $\binom{64}{8} = 4.426.165.368$. La primera mejora que podemos considerar consiste en no poner nunca más de una reina en una fila. Esto reduce la representación del tablero a un vector de ocho elementos, cada uno de los cuales da la posición de la reina dentro de la fila correspondiente. Por ejemplo, el vector (3, 1, 6, 2, 8, 6, 4, 7) representa la situación en que la reina de la fila 1 está en la columna 3, la reina de la fila 2 está en la columna 1, y así sucesivamente. Esta situación concreta no es una solución para nuestro problema, puesto que las reinas de las filas 3 y 6 se encuentran en la misma columna, y además hay dos pares de reinas en la misma diagonal. Empleando esta representación, podríamos escribir un algoritmo empleando ocho bucles anidados (para la función *solución*, véase el Problema 9.43):

```

programa reinas1
  para  $i_1 \leftarrow 1$  hasta 8 hacer
    para  $i_2 \leftarrow 1$  hasta 8 hacer
      ...
      para  $i_8 \leftarrow 1$  hasta 8 hacer
         $sol \leftarrow [i_1, i_2, \dots, i_8]$ 
        si solución(sol) entonces escribir sol
      parar
  escribir «no hay solución»

```

El número de situaciones que hay que considerar se ha reducido a $8^8 = 16.777.216$, aunque de hecho el algoritmo se detiene después de considerar solamente 1.299.852 situaciones.

La representación del tablero mediante un vector nos impide intentar poner dos reinas en una misma fila. Una vez que nos damos cuenta de esto, es natural ser igualmente sistemáticos en lo tocante a las columnas. Por tanto, ahora representaremos el tablero mediante un vector formado por ocho números *diferentes* entre 1 y 8, esto es, mediante una permutación de los ocho primeros números enteros. Esto produce el siguiente algoritmo:

```

programa reinas2
  sol  $\leftarrow$  permutación inicial
  mientras sol  $\neq$  permutación final y no solución(sol) hacer
    sol  $\leftarrow$  permutación siguiente
  si solución(sol) entonces escribir sol
    sino escribir «no hay solución»

```

Hay varias maneras naturales de generar sistemáticamente todas las permutaciones de los n primeros números enteros. Por ejemplo, podemos poner cada valor correspondiente en la primera posición, y generar recursivamente, para cada valor inicial, todas las permutaciones de los $n-1$ elementos restantes. El procedimiento siguiente muestra la manera de hacer esto. Aquí $T[1..n]$ es una matriz global a la que se da como valor inicial $[1, 2, \dots, n]$, y la llamada inicial al procedimiento es $perm(1)$. Esta forma de generar permutaciones es también una especie de vuelta atrás.

```

procedimiento perm(i)
  si  $i = n$  entonces usar(T) {T es una nueva permutación}
    sino para  $j \leftarrow i$  hasta  $n$  hacer intercambiar  $T[i]$  y  $T[j]$ 
      perm(i+1)
      intercambiar  $T[i]$  y  $T[j]$ 

```

Este enfoque reduce el número de situaciones posibles a $8! = 40.320$. Si se utiliza el algoritmo precedente para generar las permutaciones, entonces en realidad solo se consideran 2.830 situaciones antes de que el algoritmo encuentre una solución. Es más complicado generar las permutaciones que generar todos los posibles vectores de ocho enteros entre 1 y 8. Por otra parte, resulta más sencillo verificar en este caso si una situación dada es una solución. Dado que ya sabemos que dos reinas no pueden estar ni en la misma fila ni en la misma columna, basta verificar que no estén en la misma diagonal.

Partiendo de un método muy grosero que ponía reinas absolutamente en todos los lugares del tablero, hemos progresado primero hasta un método que nunca pone dos reinas en la misma fila, y después a otro mejor que sólo considera aquellas situaciones en que dos reinas nunca pueden estar ni en la misma fila ni en la misma columna. Sin embargo, todos estos algoritmos comparten un defecto común: nunca comprueban si una situación es una solución mientras no se hayan colocado todas las reinas en el tablero. Por ejemplo, incluso el mejor de ellos hace 720 intentos inútiles para colocar las seis últimas reinas en el tablero cuando ha comenzado por poner las dos primeras en la diagonal principal, lugar en que desde luego se amenazan una a otra.

La vuelta atrás nos permite mejorar esto. Como primer paso, reformulamos el problema de las ocho reinas como un problema de búsqueda en un árbol. Decimos que un vector $V[1..k]$ de enteros entre 1 y 8 es k -prometedor, para $0 \leq k \leq 8$, si ninguna de las k reinas colocadas en las posiciones $(1, V[1]), (2, V[2]), \dots, (k, V[k])$ amenaza a ninguna de las otras. Matemáticamente, un vector V es k -prometedor si, para todo par de enteros i y j entre 1 y k , con $i \neq j$, tenemos que $V[i] - V[j] \notin \{i - j, 0, j - i\}$. Pa-

ra $k \leq 1$, todo vector V es k -prometedor. Las soluciones del problema de las 8 reinas se corresponden con aquellos vectores que son 8-prometedores.

Sea N el conjunto de vectores k -prometedores, $0 \leq k \leq 8$. Sea $G = \langle N, A \rangle$ el grafo dirigido tal que $(U, V) \in A$ si y sólo si existe un entero k , con $0 \leq k < 8$, tal que

- ◊ U es k -prometedor
- ◊ V es $(k + 1)$ -prometedor, y
- ◊ $U[i] = V[i]$ para todo $i \in [1..k]$

Este grafo es un árbol. Su raíz es el vector vacío correspondiente a $k = 0$. Sus hojas son o bien soluciones ($k = 8$) o posiciones sin salida ($k < 8$) tales como $[1, 4, 2, 5, 8]$: en tal situación, resulta imposible colocar una reina en la fila siguiente sin amenazar por lo menos a una de las reinas que ya están en el tablero. Las soluciones del problema de las ocho reinas se pueden obtener explorando este árbol. Sin embargo, no generamos explícitamente el árbol para explorarlo después. Más bien, se generan y abandonan los nodos en el transcurso de la exploración. El recorrido en profundidad es el método evidente, sobre todo si sólo necesitamos una solución.

Esta técnica posee dos ventajas con respecto al algoritmo que va probando sistemáticamente todas las permutaciones. En primer lugar, el número de nodos del árbol es menor que $8! = 40.320$. Aunque no resulta sencillo calcular teóricamente este número, es fácil contar los nodos empleando una computadora: hay 2.057. De hecho, basta con explorar 114 nodos para obtener una primera solución. En segundo lugar, para decidir si un vector es k -prometedor, sabiendo que es una extensión de un vector $(k-1)$ -prometedor, sólo necesitamos comprobar la última reina que haya que añadir. Esto se puede acelerar si asociamos a cada nodo prometedor el conjunto de columnas, el de diagonales positivas (a 45 grados) y el de diagonales negativas (a 135 grados) controlados por las reinas que ya están puestas.

En el procedimiento siguiente, $sol[1..8]$ es una matriz global. Para imprimir todas las soluciones del problema de las ocho reinas, se llama a $reinas(0, \emptyset, \emptyset, \emptyset)$:

procedimiento $reinas(k, col, diag45, diag135)$

$\{sol[1..k]$ es k -prometedor,

$col = \{sol[i] \mid 1 \leq i \leq k\}$,

$diag45 = \{sol[i] - i + 1 \mid 1 \leq i \leq k\}$ y

$diag135 = \{sol[i] + i - 1 \mid 1 \leq i \leq k\}$ }

si $k = 8$ **entonces** {un vector 8-prometedor es una solución}

escribir sol

sino {explorar las extensiones $(k+1)$ -prometedoras de sol }

para $j \leftarrow 1$ **hasta** 8 **hacer**

si $j \notin col$ y $j - k \notin diag45$ y $j + k \notin diag135$

entonces $sol[k + 1] \leftarrow j$

$\{sol[1..k+1]$ es $(k+1)$ -prometedor}

$reinas(k+1, col \cup \{j\}$,

$diag45 \cup \{j - k\}, diag135 \cup \{j + k\})$

Está claro que el problema se puede generalizar a un número arbitrario de reinas: ¿cómo se pueden situar n reinas en un «tablero» $n \times n$ de tal manera que ninguna de ellas amenace a ninguna de las demás? Tal como cabe esperar, la ventaja obtenida al utilizar la vuelta atrás en lugar de un enfoque exhaustivo se vuelve más pronunciada a medida que crece n . Por ejemplo, para $n = 12$ son 479.001.600 las posibles permutaciones que hay que considerar. Empleando el generador de permutaciones dado anteriormente, la primera solución que se encuentra corresponde a la 4.546.044-ésima situación examinada. Por otra parte, el árbol explorado por el algoritmo de vuelta atrás contiene solamente 856.189 nodos, y se obtiene una solución al visitar el nodo 262. El problema se puede generalizar todavía más colocando las «reinas» en tres dimensiones en un tablero $n \times n \times n$; véase el Problema 9.49.

9.6.3 El caso general

Los algoritmos de vuelta atrás se pueden utilizar aun cuando las soluciones buscadas no tengan todas necesariamente la misma longitud. Véase el esquema general:

```

procedimiento vueltaatrás( $v[1..k]$ )
    { $v$  es un vector  $k$ -prometedor}
    si  $v$  es una solución entonces escribir  $v$ 
    sino para cada vector  $(k+1)$ -prometedor  $w$ 
        tal que  $w[1..k] = v[1..k]$ 
            hacer vueltaatrás( $w[1..k+1]$ )
  
```

El **sino** debería estar presente si y sólo si resulta imposible disponer de dos soluciones diferentes tales que una sea prefijo de la otra.

Tanto el problema de la mochila como el problema de las n reinas se resolvían empleando una búsqueda en profundidad en el árbol correspondiente. Algunos problemas que se pueden formular en términos de explorar un grafo implícito tienen la propiedad consistente en que corresponden a un grafo infinito. En este caso puede ser necesario emplear un recorrido en anchura para evitar la exploración interminable de alguna rama infinita carente de resultados positivos. El recorrido en anchura también es apropiado si tenemos que encontrar una solución comenzando a partir de una cierta situación inicial y efectuando el menor número de pasos posible.

9.7 RAMIFICACIÓN Y PODA

Al igual que la vuelta atrás, la ramificación y poda es una técnica para explorar un grafo dirigido implícito. Una vez más, este grafo es normalmente acíclico, o incluso un árbol. Esta vez vamos a buscar la solución óptima de algún problema. En cada nodo, calculamos una cota del posible valor de aquellas soluciones que pudieran encontrarse más adelante en el grafo. Si la cota muestra que cualquiera de estas soluciones tiene que ser necesariamente peor que la mejor solución hallada hasta el momento, entonces no necesitamos seguir explorando esta parte del grafo.